

大模型内容安全与质量评估智能体 · 研发日报

日期	2026年7月7日（周二）
课题	内容保真度与治理质量评估智能体（LLM-as-Judge）
阶段	Phase 6 — 三层检测架构 + 算法修正 + 用例验证
日报编号	Day 8 / 10
组长	李首澎
组员	阿思晗、由靖喆

一、组员进度汇报

1.1 组员：阿思晗

今日任务：典型用例准备 + 瑕疵合并逻辑 + 用例验证

上午 — 3 组典型用例准备

准备了 3 组典型用例，覆盖系统的三类核心评估场景：

1. 用例 A — 优秀治理：治理后文本仅进行标点规范化和段落对齐，内容保真度高。预期：overall_score >= 0.8，verdict = pass，无 critical 瑕疵。
2. 用例 B — 过度清洗：治理后文本删除了原文约 35% 的内容，包括关键数据说明。预期：overall_score < 0.5，verdict = fail，over_clean 维度扣分严重。
3. 用例 C — 严重误改：治理后文本中关键数值被篡改（如营业收入 120 万 -> 12 万）。预期：overall_score < 0.5，verdict = fail，mis_edit 维度扣分严重。

下午 — 瑕疵合并逻辑 + 用例验证

实现了 detect_flaws() 的瑕疵合并逻辑：同一 category+severity 的多条瑕疵合并为一条，避免"表格->文本"等场景产生大量重复 flaw。随后对 3 组用例进行了独立运行验证，确认评估结果与预期一致。

任务	完成状态
用例 A — 优秀治理文本对 + 预期结果	✅ 已完成
用例 B — 过度清洗文本对 + 预期结果	✅ 已完成
用例 C — 严重误改文本对 + 预期结果	✅ 已完成
瑕疵合并逻辑实现	✅ 已完成
用例运行验证	✅ 已完成

计划工时：4h 实际工时：约 4h 偏差：持平

1.2 组员：由靖喆

今日任务：前端交互优化 + 用例环境验证

上午 — 前端交互优化

对系统前端进行交互优化，改进了输入框的 placeholder 提示、加载状态显示和错误信息展示。

下午 — 用例环境验证

搭建演示环境：确认 Streamlit 应用在 localhost:8501 正常启动、FastAPI 后端在 localhost:8081 正常响应。对 3 组用例进行了独立运行验证，确认评估结果与预期一致。

任务	完成状态
前端交互优化	✅ 已完成
演示环境搭建和验证	✅ 已完成
用例运行验证	✅ 已完成

计划工时：2h 实际工时：约 2h 偏差：持平

二、组长汇总

2.1 今日整体进度说明

Day 8 的核心任务是实现三层检测架构和算法修正机制。今日完成了三层结构检测架构（Diff -> LLM -> Merge）的设计与实现、`algorithm_adjustment` 字段的新增、同类瑕疵合并逻辑，以及 3 组典型用例的准备和验证。

2.2 组员任务完成情况汇总

姓名	今日任务	完成状态	备注
李首澎（组长）	三层检测架构 + 算法修正记录 + 用例设计	✅ 全部完成	核心模块
阿思晗	用例准备 + 瑕疵合并逻辑 + 用例验证	✅ 全部完成	模块 + 验证
由靖喆	前端优化 + 环境搭建 + 用例验证	✅ 全部完成	前端 + 环境

2.3 组长今日工作内容

1. 三层结构检测架构：设计并实现了"Diff 检测 -> LLM 评分 -> Merge 融合"三层架构。第一层（Diff）通过确定性规则检测 Markdown 表格破坏、列表压缩等结构变化；第二层（LLM）负责语义理解和维度评分；第三层（Merge）将算法检测结果与 LLM 输出融合，当算法检测到 critical 级别瑕疵但 LLM 漏检时，通过 `algorithm_adjustment` 修正维度分数。
2. 算法修正记录（`algorithm_adjustment`）：在评估输出中新增 `algorithm_adjustment` 字段，保留 LLM 原始维度分数，单独记录算法修正值和修正原因。最终分数基于修正后的维度计算，但 LLM 原始输出完整保留，便于溯源和解释。

3. **典型用例设计**：设计用例 A（优秀治理）、用例 B（过度清洗）、用例 C（严重误改），准备治理前后文本对和预期评估结果。

计划工时：5h 实际工时：约 5h 偏差：持平

三、今日研发过程记录

3.1 三层检测架构

第一层：Diff 检测（确定性规则）

- 表格/列表结构丢失检测
- 段落级相似度分析（SequenceMatcher）
- 字符级 diff（数值变更、删除、修改）

↓

第二层：LLM 评分（语义理解）

- 四维度评分（semantic/factual/structure/readability）
- 瑕疵检出（category + severity + location）

↓

第三层：Merge 融合

- 算法检测到 critical 但 LLM 漏检 -> algorithm_adjustment 修正
- LLM 原始分数保留，修正值单独记录

3.2 三层检测架构核心代码

第一层：确定性规则检测（chain.py:812-839）

```
# 表格结构丢失检测
before_pipe = anchored_before.count("|")
after_pipe = anchored_after.count("|")
has_table_separator = "|---" in anchored_before
table_detected = before_pipe >= 6 and has_table_separator and after_pipe < 2
if table_detected:
    flaws.append({"type": "structure_loss", "category": "structure", "severity":
"critical"})

# 有序列表压缩检测
before_list_items = len(re.findall(r"(?:^|\n)\s*\d+[\.\.])\s", anchored_before))
after_list_items = len(re.findall(r"(?:^|\n)\s*\d+[\.\.])\s", anchored_after))
if before_list_items >= 3 and after_list_items == 0:
    flaws.append({"type": "structure_loss", "category": "structure", "severity":
"major"})
```

第二层：段落级相似度分析（chain.py:879-930）

```
ratio = difflib.SequenceMatcher(None, b_text, a_text).ratio()

if ratio > 0.85:
```

```

# 轻微调整, 检查数字变化
changes = _char_diff(b_text, a_text)
for ch in changes:
    b_nums = num_pattern.findall(ch.get("before_text", ""))
    a_nums = num_pattern.findall(ch.get("after_text", ""))
    if b_nums and a_nums and b_nums != a_nums:
        # 数值变更, 标为 mis_edit
        flaws.append({"category": "mis_edit", "severity": "major"})
elif ratio < 0.3:
    # 大幅改写, 标为 critical
    flaws.append({"category": "over_clean", "severity": "critical"})

```

第三层: Merge 融合 — 算法修正 (chain.py:1191-1232)

```

# Diff 检测到 critical structure → structure 分数强制降到 0.3
if df_sev == "critical" and df_cat == "structure":
    llm_struct = next((d.score for d in dimensions if d.dimension == "structure"),
0.5)
    if llm_struct > 0.5:
        algo_adjustments["structure"] = {
            "llm_score": llm_struct,
            "penalty": -0.5,
            "adjusted_score": 0.3,
            "reason": "Markdown 表格/列表结构被破坏 (算法检测)",
        }

# Diff 检测到 critical omission → semantic 扣 0.3
elif df_sev == "critical" and df_cat in ("omission", "over_clean"):
    llm_semantic = next((d.score for d in dimensions if d.dimension ==
"semantic"), 0.5)
    if llm_semantic > 0.6:
        algo_adjustments["semantic"] = {
            "llm_score": llm_semantic,
            "penalty": -0.3,
            "adjusted_score": max(0.0, llm_semantic - 0.3),
            "reason": "大量内容被删除 (算法检测)",
        }

# LLM 原始分数保留在 dimensions, 修正值单独记录在 algorithm_adjustment
for d in dimensions:
    if d.dimension in algo_adjustments:
        adj = algo_adjustments[d.dimension]
        scoring_dimensions.append(DimensionScore(
            dimension=d.dimension, score=adj["adjusted_score"], ...
        ))
    else:
        scoring_dimensions.append(d)

```

3.3 算法修正机制

当 Diff 检测到 critical 级别的结构/遗漏瑕疵，但 LLM 的对应维度分数偏高时，算法自动修正：

触发条件	修正维度	修正幅度	原因
Diff 检测到 critical structure	structure	-0.2	Markdown 表格/列表结构被破坏
Diff 检测到 critical omission	semantic	-0.3	大量内容被删除

修正后的分数和原始分数都记录在 `algorithm_adjustment` 字段中，确保可追溯。

3.3 典型用例设计

用例	场景	预期评估结果
A	优秀治理	score >= 0.8, pass
B	过度清洗（删除 35%）	score < 0.5, fail, over_clean
C	严重误改（120万->12万）	score < 0.5, fail, mis_edit

四、关键产出

产出内容	对应任务	说明
三层检测架构	核心模块	Diff -> LLM -> Merge 三层融合
<code>algorithm_adjustment</code> 字段	核心模块	LLM 原始分数 + 算法修正值 + 修正原因
瑕疵合并逻辑	核心模块	同 category+severity 合并，避免重复
3 组典型用例	验证	优秀治理 / 过度清洗 / 严重误改

五、当前进度评估

5.1 里程碑进度看板

里程碑	完成标志	目标日	当前状态	风险等级
M1-M3	已达标	Day 6	✅ 已完成	低
M4: 材料完备	用例 + Demo 就绪	Day 9	准备中	低
M5: 最终提交	全部材料打包提交	Day 10	未开始	低

5.2 总体进度状态

当前状态：进度正常，三层检测架构已实现，典型用例已验证通过。

六、后续安排

后续工作将进行全指标最终复核、性能优化和验收材料打包。

七、总结

今天完成了三层检测架构（Diff -> LLM -> Merge）的设计与实现，新增了 `algorithm_adjustment` 字段保留算法修正记录，实现了同类瑕疵合并逻辑。3 组典型用例验证通过，系统核心功能已完备。